Module 7 Week B — Reading: Pre-Trained Inference for QA & Summarization

Everything in this reading is fair game for readiness checks, drills, and labs. A learner who reads this carefully should be able to complete the Week B drill and lab without attending the concept session.

What success looks like: After this reading, you can construct a Hugging Face `pipeline()` object for two tasks (extractive question answering and abstractive summarization); explain why an encoder hosts an extractive Question Answering (QA) head and an encoder–decoder hosts a summarization head; evaluate QA outputs with exact-match and token-F1, and summaries with ROUGE; and reason about when fine-tuning is worth its cost relative to using a pre-trained model directly.

In This Reading

1. The `pipeline()` Abstraction
2. Extractive QA: Span Prediction Mechanics
3. Calling a QA Pipeline
4. Evaluating Extractive QA: EM and Token-F1
5. Encoder–Decoder Architecture in Practice
6. Calling a Summarization Pipeline
7. Evaluating Summarization: ROUGE
8. Fine-Tuning vs. Pre-Trained Inference: The Decision
9. Production Shape: Where This All Lands

- Troubleshooting
- Key Terms
- Summary
- What's Next

---

1. The `pipeline()` Abstraction

In Week A you used `transformers.AutoModelForSequenceClassification` and `transformers.Trainer` directly. You constructed a tokenizer, a model, training arguments, and a metrics function — every step of the pipeline was explicit. Week A's level of explicitness is necessary when you are *training* a model; you need to control every knob.

For *inference* on common tasks, the Hugging Face library provides a higher-level convenience layer: `pipeline()`. A pipeline is a single callable that bundles together everything needed to take raw input (text) and produce task-shaped output (a label, an answer span, a generated summary).

A pipeline contains three things:

1. **A tokenizer** — converts strings into model inputs (`input_ids`, `attention_mask`).
2. **A model** — loaded with the right task-specific head for the chosen task (sequence classification, question answering, summarization, etc.).
3. **Post-processing logic** — converts raw model outputs (logits, generated token IDs) into the final task output (label string, answer span, generated text).

You construct a pipeline by naming the task and (optionally) the model:

```python
from transformers import pipeline

qa = pipeline("question-answering", model="distilbert-base-cased-distilled-squad"
summ = pipeline("summarization", model="sshleifer/distilbart-cnn-6-6")
```

The first argument is the **task name**. Hugging Face supports many: `"sentiment-analysis"`, `"text-classification"`, `"question-answering"`, `"summarization"`, `"translation"`, `"text-generation"`, and more. Each task name implies the right head architecture and the right post-processing.

The second argument is the **model id** on Hugging Face Hub. You can also omit it; Hugging Face will pick a default model for the task (e.g., `distilbert-base-uncased-finetuned-sst-2-english` for sentiment). For reproducibility — especially in evaluations — name the model explicitly. In Week B, we always name the model.

When to use a pipeline vs. drop down to AutoModel

A pipeline is the right tool when:

- You are doing inference (not training).
- The task is one of the standard supported tasks.
- You can accept the default decoding / extraction logic.

You drop down to `AutoModelFor<Task>` directly when:

- You need custom batching (the pipeline batches one input at a time by default).
- You need custom decoding logic (e.g., constrained generation, custom span constraints).
- You are going to fine-tune the model (then you also need `Trainer` or a manual loop).

For the lab and integration in Week B, the pipeline is the right tool. The Honors Track Tuesday stretch will drop down to `AutoModelForQuestionAnswering` to build a manual QA inference loop — that is the pedagogical contrast.

---

2. Extractive QA: Span Prediction Mechanics

Extractive question answering is a different task shape than the sequence classification you fine-tuned in Week A. Both tasks use an encoder transformer (DistilBERT in our examples), but they use a different head and produce a different output.

The QA task

Given a **question** (a string, like "*Where was Alan Turing born?*") and a **context** (a passage of text that contains the answer), an extractive QA model returns a **contiguous substring of the context** that answers the question. "Extractive" means the answer must literally appear in the context — the model does not generate text or invent facts. If the context says "*Alan Turing was born in Maida Vale, London,*" the model returns the substring "Maida Vale, London" (or "Maida Vale" — span boundaries are part of what the model has to learn).

This is a feature, not a limitation. An extractive QA model **cannot hallucinate** — it can only return text that is actually in the document. In production, this property is valuable for grounded question answering over verified sources (medical literature, legal filings, internal documentation).

How extractive QA works under the hood

The model concatenates the question and the context with a separator token: `[CLS] question [SEP] context [SEP]`. It runs this combined sequence through the encoder, producing a hidden state per token. Then a **QA head** — two small linear layers — produces two logits per token over the context portion: a **start logit** and an **end logit**.

The predicted answer span is the pair `(i, j)` (with `i ≤ j`, both pointing into the context tokens) that maximizes `start_logit[i] + end_logit[j]`, subject to a `max_answer_length` constraint (typically 30 tokens).

You don't compute this by hand — the pipeline handles it. But the mental model matters because it tells you:

- **Why encoder-family models host extractive QA heads.** The encoder produces per-token contextual representations; the QA head consumes them. A pure decoder model has no per-token representations of the input in the same shape.
- **Why the answer is always a substring.** The output is two indices into the context tokens. Whatever lies between is the answer.
- **Why long contexts are a challenge.** The model has a maximum sequence length (typically 384 or 512 tokens for BERT-family models). A context longer than that has to be windowed (split into overlapping chunks, each scored separately, with the highest-scoring span returned).

Public extractive QA models on Hugging Face Hub

Model	Size	Notes
`distilbert-base-cased-distilled-squad`	~65M params	Distilled from a BERT fine-tuned on SQuAD (Stanford Question Answering Dataset) v1.1. CPU-feasible. **Our Week B pick.**
`bert-large-uncased-whole-word-masking-finetuned-squad`	~340M params	Strong performance but slow on CPU.
`deepset/roberta-base-squad2`	~125M params	Trained on SQuAD v2.0; supports a "no answer" prediction when the context doesn't contain an answer.

We pick `distilbert-base-cased-distilled-squad` because it is fast on CPU, well-documented, and trained on SQuAD v1.1 (a public extractive-QA benchmark; always has an answer in the context — keeping the evaluation surface clean for Week B).

---

3. Calling a QA Pipeline

The QA pipeline call looks like this:

```python
from transformers import pipeline

qa = pipeline("question-answering", model="distilbert-base-cased-distilled-squad"

context = (
    "Alan Mathison Turing was an English mathematician, logician, "
    "and computer scientist. He was born in Maida Vale, London, on 23 June 1912.
    "Turing is widely considered to be the father of theoretical computer science
)
question = "Where was Alan Turing born?"

result = qa(question=question, context=context)
print(result)
# {"answer": "Maida Vale, London", "score": 0.83, "start": 110, "end": 128}
```

The result is a dictionary with four fields:

- `answer` — the extracted span as a string.
- `score` — the joint start+end probability. A confidence signal but **not** a calibrated probability; treat it as a relative ordering, not a guarantee.
- `start`, `end` — **character** offsets into the context string (not token indices). Useful for highlighting the answer in the original text. Note: Section 2 described `(i, j)` token indices over the context — that is the model's internal mechanism; the pipeline post-processes those token spans back into character offsets before returning the result.

Edge cases

A few cases the pipeline will surprise you with if you don't anticipate them:

- **The pipeline always returns an answer**, even when the context doesn't contain one. With a SQuAD v1.1–trained model like the one we use in Lab 7B (`distilbert-base-cased-distilled-squad`), every query produces a span. Use the `score` field as a confidence signal: very low scores often correspond to the model "guessing." For SQuAD v2.0–trained models like `deepset/roberta-base-squad2`, the empty string `""` is the explicit "no answer" prediction.
- **Long contexts get windowed.** If the context exceeds the model's max length, the pipeline automatically splits it into overlapping windows, runs each, and returns the highest-scoring span across windows. This is mostly transparent, but the long-context score is the *max* across windows, which can favor confidently-wrong spans in irrelevant chunks. The Honors Track Thursday stretch explores this trade-off.
- **The first call downloads the model.** `pipeline("question-answering", model=...)` will download ~250 MB the first time. Don't worry that the script seems to hang — that's the download. Subsequent runs use the cached weights.

---

4. Evaluating Extractive QA: EM and Token-F1

Once you have predicted answers, you need to score them against gold answers. QA evaluation has two standard metrics: **exact-match (EM)** and **token-F1**. Both are reported together because they capture different things.

Normalization

Both metrics depend on **normalization**. If the gold answer is "Maida Vale, London" and the prediction is "Maida vale, London.", you do not want to score that as wrong — the difference is purely cosmetic. The standard SQuAD normalization function does four things:

1. **Lowercase** the string.
2. **Strip articles** — remove standalone occurrences of `a`, `an`, `the` (using word-boundary regex so `the` in `there` is not stripped).
3. **Strip punctuation** — remove all `string.punctuation` characters.
4. **Collapse whitespace** — replace runs of whitespace with a single space; strip leading/trailing whitespace.

In Python:

```python
import re
import string

def normalize_answer(s: str) -> str:
    s = s.lower()
    s = re.sub(r"\b(a|an|the)\b", " ", s)
    s = "".join(ch for ch in s if ch not in string.punctuation)
    s = " ".join(s.split())
    return s
```

Normalization is part of the metric. Comparing un-normalized strings will systematically under-report both EM and F1. The drill exercises this.

Exact-match (EM)

EM is the strictest metric. After normalization, EM is `1` if `predicted == gold`, else `0`. Aggregate EM is the fraction of questions where EM is 1.

EM is unforgiving. A prediction of "Maida Vale London" against gold "Maida Vale, London" — without comma — is EM = 0 if you forgot to strip punctuation in normalization. EM = 1 if you remembered. EM punishes off-by-one tokens, partial answers, and answer-padding all equally.

Token-F1

Token-F1 captures partial credit. After normalization, treat the predicted answer and the gold answer as **bags of tokens** (split on whitespace). Compute:

- **Token precision:** how many predicted tokens appear in gold? `overlap / |predicted|`.
- **Token recall:** how many gold tokens appear in predicted? `overlap / |gold|`.
- **Token-F1:** harmonic mean of the two.

Aggregate token-F1 is the average over questions.

Edge cases:

- If both predicted and gold are empty (after normalization), F1 = 1.0 by convention (only relevant for SQuAD v2.0 no-answer predictions).
- If exactly one is empty, F1 = 0.0 (do not return NaN — that breaks aggregations).
- If the two strings have **no overlap**, F1 = 0.0.

Why both metrics are reported

A model that gets EM = 0.65 and F1 = 0.85 has many partial-credit answers — it is finding the right region but stumbling on span boundaries. A model that gets EM = 0.70 and F1 = 0.74 is essentially binary — when it is right it is right, when it is wrong it is completely wrong. The gap between EM and F1 is itself a diagnostic signal.

Methodology specification

When you report QA numbers, you must state explicitly:

- The normalization function used (SQuAD-style is the default; document any deviation).
- How multiple gold answers are handled (SQuAD provides multiple acceptable golds per question; the standard is to take the **max** EM/F1 across golds).
- How no-answer questions are scored (SQuAD v2.0; not relevant for Week B's v1.1 evaluation).

The Lab will require you to document these choices in your `qa-evaluation-report.md`.

---

5. Encoder–Decoder Architecture in Practice

In Week A's reading, we introduced the three transformer architectural families: encoder, decoder, encoder–decoder. Week A used encoders (DistilBERT) for classification. Week B revisits the encoder–decoder family with a hands-on summarization task.

Recall the structural difference. An **encoder–decoder** transformer has two stacks:

- The **encoder** reads the input and produces a sequence of hidden states (just like in Week A's encoder-only models).
- The **decoder** generates the output autoregressively — one token at a time — and at each step it attends both to the tokens it has already generated *and* to the encoder's hidden states. This is called **cross-attention**.

This is the right architecture when input and output are *different sequences*. In summarization, the input is a long article and the output is a short summary. They are both text but differently shaped. Translation, generative QA, and speech-to-text follow the same pattern.

Canonical encoder–decoder models

Model	Size	Notes
`t5-small`	~60M params	T5 frames every task as text-to-text. Versatile but produces shorter, more conservative outputs.
`t5-base`	~220M params	Larger, slower on CPU.
`facebook/bart-base`	~140M params	BART is pre-trained as a denoising autoencoder; strong on summarization.
`facebook/bart-large-cnn`	~400M params	BART fine-tuned on CNN/DM (CNN/Daily Mail) news summarization. High quality, slow on CPU.
`sshleifer/distilbart-cnn-6-6`	~230 MB	Distilled BART-CNN. ~60% faster than the full model, retains ~95% of ROUGE quality. **Our Week B pick.**

We pick `sshleifer/distilbart-cnn-6-6` because it is fast on CPU and trained on news articles (similar genre to our tech news corpus). **Known quirk:** this checkpoint tends to repeat n-grams in its outputs. Set `no_repeat_ngram_size=3` in your generation call as a default for this model — the troubleshooting section and the call example in Section 6 show this.

Generation control

Unlike classification or extractive QA, encoder–decoder generation has many knobs that change the output:

- `max_length` — the maximum number of tokens in the output. Setting this too small truncates mid-sentence; too large produces padding-like rambling.
- `min_length` — the minimum number of tokens. Forces the model to keep generating even when it would naturally stop.
- `num_beams` — the width of beam search. Beam search keeps the top-N candidate sequences at each step and picks the best at the end. Higher = better quality but linearly more compute. Default for BART summarization is 4.
- `do_sample` — `False` (the default for summarization) gives deterministic beam search. `True` enables stochastic sampling, controlled by `top_k` and `top_p`.
- `length_penalty` — > 1.0 favors longer outputs; < 1.0 favors shorter. Used to balance against `min_length` / `max_length`.
- `no_repeat_ngram_size` — prevents the model from generating the same n-gram twice in the same output (a common failure mode in greedy decoding).

For evaluation reproducibility, **always use `do_sample=False`** — sampling produces different summaries on each run, which makes your ROUGE numbers unreproducible.

---

6. Calling a Summarization Pipeline

The summarization pipeline follows the same shape as the QA pipeline:

```python
from transformers import pipeline

summ = pipeline("summarization", model="sshleifer/distilbart-cnn-6-6")

article = (
    "Researchers at Stanford University reported on Tuesday that a new battery te
    "based on solid-state lithium-metal cells has achieved 1,000 charge cycles in
    "tests with less than 5% capacity loss. The breakthrough could enable electri
    "with both longer range and faster charging, addressing two of the most persi
    "barriers to widespread adoption. The research team plans to license the tech
    "to a commercial battery manufacturer for production trials in 2027."
)

out = summ(article, max_length=80, min_length=20, do_sample=False, num_beams=4, r
print(out)
# [{"summary_text": "Researchers at Stanford report a new solid-state lithium-met
#                   achieving 1,000 cycles with under 5% capacity loss, potentia
#                   longer-range and faster-charging EVs."}]
```

A few things to notice:

- The pipeline returns a **list of dictionaries** (one per input). When you pass a single string, it is wrapped in a list of length one. To get the summary string, you index `out[0]["summary_text"]`.
- The output length is in **tokens**, not words or characters. A `max_length=80` may correspond to roughly 60 words, depending on tokenization.
- The output is **abstractive** — the model can produce words that are not in the input. In the example above, "EVs" is in the output but "EVs" was not in the input ("electric vehicles" was).

Edge cases

- **Long inputs get truncated.** BART has an input max length around 1024 tokens. Longer articles are silently truncated. If your article is much longer than this, consider summarizing in chunks and concatenating, or use a long-context summarizer.
- **`min_length` too aggressive forces rambling.** If `min_length=80` and the article only has ~40 tokens of summarizable content, the model will pad with awkward repetitions or made-up details.
- **First call downloads the model.** ~230 MB for distilled BART. Be patient on the first run.

---

7. Evaluating Summarization: ROUGE

Summarization evaluation is harder than classification or extractive QA. There is no single "correct" summary — many different summaries can be acceptable. The standard approach is **lexical overlap with one or more reference summaries**, captured by the **ROUGE** family of metrics.

ROUGE stands for *Recall-Oriented Understudy for Gisting Evaluation*. The original metric was recall-only; in modern usage we report the **F1** version (precision + recall harmonic mean) of three ROUGE variants:

- **ROUGE-1** — overlap of unigrams (single tokens) between predicted and reference summary.
- **ROUGE-2** — overlap of bigrams (adjacent token pairs).
- **ROUGE-L** — overlap of the longest common subsequence (LCS) between predicted and reference. LCS allows non-contiguous matches.

For each variant, precision = `overlap / |predicted|`, recall = `overlap / |reference|`, F1 = harmonic mean. Reporting F1 (not recall) is standard because it punishes a model that "summarizes" by emitting the entire input — that has perfect recall but useless precision.

Computing ROUGE in Python

The `rouge-score` library (Google Research) is the canonical implementation:

```python
from rouge_score.rouge_scorer import RougeScorer

scorer = RougeScorer(["rouge1", "rouge2", "rougeL"], use_stemmer=True)

reference = "Stanford researchers achieve 1,000-cycle solid-state battery breakth
predicted = "Researchers at Stanford report a new solid-state battery achieving 1

scores = scorer.score(reference, predicted)
print(scores["rouge1"].fmeasure)
print(scores["rouge2"].fmeasure)
print(scores["rougeL"].fmeasure)
```

Two things to notice:

- `scorer.score(reference, predicted)` — the **reference** comes first, then the **predicted**. This is the standard order. Reversing them does not produce a runtime error (the result is symmetric for F1) but reversing makes precision and recall come out swapped — bugs are hard to find.
- `use_stemmer=True` — applies Porter stemming (e.g., "running" and "runs" both reduce to "run"). The standard summarization-evaluation setting. Document any deviation.

What ROUGE catches and what it misses

ROUGE measures **lexical agreement** between summary and reference. It catches:

- Did the summary include the right entities, numbers, and key terms?
- Did it use phrasing similar to a human would use?
- Did the summary stay roughly the right length?

ROUGE does NOT catch:

- **Faithfulness.** A summary can have high ROUGE while making things up — the n-gram overlap is high if the model uses similar wording, even if the *facts* are wrong. ("Stanford researchers achieved 10,000 cycles" — wrong number, but ROUGE is barely affected because most other tokens still match.)
- **Coherence.** A summary can be word-salad of reference n-grams, syntactically broken, but still score well on ROUGE.
- **Reading order.** ROUGE-1 and ROUGE-2 are bag metrics. A scrambled-order summary scores the same as a coherent one.

This is a real production limitation. For high-stakes summarization (medical, legal, financial), ROUGE alone is not sufficient — production teams add faithfulness audits (does every claim in the summary actually appear in the source?) and human review. The Integration Task asks you to perform a small qualitative faithfulness check on three summaries you generate; this is the kind of audit that catches what ROUGE misses.

Why ROUGE is the standard anyway

ROUGE is scalable, deterministic, and correlates moderately with human judgment in aggregate. For comparing two summarization systems on the same dataset, ROUGE is fast and reproducible. The trick is to **report ROUGE as a starting point, not a verdict** — supplement it with qualitative analysis when stakes are high.

Methodology specification

When reporting ROUGE numbers, state:

- ROUGE-1, ROUGE-2, ROUGE-L F1 (always all three; ROUGE-L F1 alone is misleading).
- Tokenizer used (the `rouge-score` library default is whitespace + lowercase).
- Stemmer flag (`use_stemmer=True` for the standard setting).
- How multiple references are handled (max over references is standard).
- The model's generation parameters (`max_length`, `min_length`, `num_beams`, `do_sample`).

The Integration Task requires you to document these choices.

---

8. Fine-Tuning vs. Pre-Trained Inference: The Decision

This is the central engineering question of Week B. Week A taught you to fine-tune a transformer. Week B teaches you to evaluate a pre-trained transformer without fine-tuning. Both are valid approaches. The applied engineering question is **when to use which**.

Five factors determine the answer:

1. Labeled-data availability

Fine-tuning requires labeled examples — usually thousands. Pre-trained inference works zero-shot — no labels needed. If your domain has no labeled data and labeling is expensive (medical records requiring expert annotators, legal documents requiring lawyers), pre-trained inference is often the only feasible path. Conversely, if you have a labeled dataset already and the labels are relatively cheap to produce more of, fine-tuning is on the table.

2. Task specificity

For **generic tasks** (sentiment on news, English-language summarization), pre-trained models have very strong baselines. The lift from fine-tuning is small — maybe 2–5 percentage points. For **niche tasks** (domain-specific jargon, low-resource language, unusual label sets), pre-trained baselines can be much weaker — fine-tuning lifts performance by 10+ percentage points. The more the target task differs from what pre-training and fine-tuning data covered, the more valuable fine-tuning becomes.

3. Compute budget

Fine-tuning costs ~10–40 minutes of CPU per training run on typical student hardware (Week A's measurement; see the Lab 7A canonical times for your machine class) — and you typically need several runs to tune hyperparameters. Pre-trained inference costs a one-time download (~250 MB) and a per-inference cost measured in milliseconds. If your team has tight compute, pre-trained inference is cheaper. If you have a GPU and time, fine-tuning is feasible.

4. Iteration speed

A pre-trained pipeline can be swapped out overnight — try `distilbert-base-cased-distilled-squad`; if it underperforms, try `deepset/roberta-base-squad2`. Fine-tuning iterates on days — change a hyperparameter, re-train, re-evaluate. If your problem is exploratory and you don't yet know what good looks like, pre-trained inference lets you discover faster.

5. Quality target

If your pre-trained baseline already meets the quality bar your application requires, fine-tuning is wasted effort. If the baseline is short of the bar, fine-tuning is the next investment to consider.

Putting it together

In Week B's Integration Task, you write an **integrated evaluation report** that compares Week A's fine-tuned classifier against Week B's pre-trained QA and summarization models on the same axes. The report's central act is producing a **production decision matrix** — for each of three hypothetical deployment scenarios, would you fine-tune or use pre-trained? — and grounding the recommendation in the measured numbers.

This is the kind of decision an applied ML engineer makes weekly. There is no formula; there is judgment grounded in measurement. Week B's deliverables ask you to do the measurement and exercise the judgment.

---

9. Production Shape: Where This All Lands

A production NLP (Natural Language Processing) service typically has the following shape:

1. **A router** — for each incoming request, decide what task it is (classification, QA, summarization).
2. **One or more pre-trained pipelines** — bundled inference for common tasks. Often deployed as `pipeline()`-style objects behind an API endpoint (Module 10 will wrap them in FastAPI).
3. **One or more fine-tuned models** — for high-value or domain-specific tasks where the lift over pre-trained justified the investment.
4. **A shared evaluation harness** — periodic scoring of all production endpoints against benchmark sets, with alerts when metrics drift.
5. **A logging and monitoring layer** — Module 11 covers this.

The integrated evaluation report you produce in Integration 7B is a small version of (4). It establishes a baseline against which future models will be compared.

Forward references:

- **Module 8 (RAG).** The QA pipeline you build here is the *generator side* of a retrieval-augmented system. M8 wraps a QA pipeline behind a retriever (Weaviate vector search) — the retriever finds relevant passages, the QA pipeline answers questions from them. Same pipeline, different inputs.
- **Module 10 (Deployment).** Pre-trained pipelines are wrapped as FastAPI endpoints — a single function that accepts a request and returns a JSON response. The `pipeline()` object is the natural unit for this.
- **Module 11 (Monitoring).** Periodic evaluation harnesses, like the one you build in Week B, become continuous evaluation pipelines that run on a schedule.

The skills you build this week — pipelining, evaluation methodology, the cost/quality decision — are the foundations of all the deployment work that follows.

Troubleshooting

"`pipeline()` is downloading something for 5 minutes." The first call to a pipeline downloads the model weights — ~250 MB for `distilbert-base-cased-distilled-squad` and ~230 MB for `sshleifer/distilbart-cnn-6-6`. Subsequent calls use the cached weights. If you have very slow internet, the first run can take 5–10 minutes; this is expected.

"My QA model returns answers that are obviously wrong." Three common causes: (a) the context doesn't actually contain the answer (the model still returns a span — check the `score` field), (b) the question is ambiguous and the model picked a different valid interpretation, (c) the context exceeds the model's max length and the answer is in the part that got truncated. The `start` and `end` fields tell you where in the context the model found the answer — you can use them to debug.

"My ROUGE numbers are zero." Almost always a tokenizer / argument-order bug. Verify (a) `scorer.score(reference, predicted)` — reference first, (b) `use_stemmer=True` is enabled, (c) the predicted summary is non-empty. If reference and predicted both look reasonable and ROUGE is still zero, print one (reference, predicted) pair and check by eye whether they share *any* tokens. If they do, your scorer construction is wrong.

"My summarization output looks repetitive or rambling." This is a well-documented characteristic of `sshleifer/distilbart-cnn-6-6` — without an n-gram repetition guard it often emits the same 3-gram several times. Tune the generation parameters: set `no_repeat_ngram_size=3` (recommended as a default for this checkpoint), lower `max_length`, keep `do_sample=False` for evaluation reproducibility. If the output is consistently bad, your `min_length` may be too aggressive — try lowering it.

"I see a warning like `do_sample=False is deprecated when num_beams > 1`." Harmless. In `transformers ≥ 4.45`, deterministic beam search is implied whenever `num_beams > 1`, so `do_sample=False` is now redundant. We keep it in the example calls because it makes the methodology explicit (and earlier transformers versions still require it). Your output is unchanged; ignore the warning.

"I get `OSError: distilbert-base-cased-distilled-squad does not appear to be a file...`." Hugging Face Hub model id may be misspelled or you have no internet. Check the model id on Hugging Face Hub; verify connectivity.

Key Terms

- **Pipeline** (`pipeline()`) — Hugging Face's high-level inference abstraction; bundles tokenizer + model + post-processing.
- **Extractive QA** — Question answering where the answer is a contiguous substring of the context.
- **Span prediction** — The technical mechanism behind extractive QA: per-token start and end logits over the context.
- `AutoModelForQuestionAnswering` — The encoder + QA head class for extractive QA.
- **Abstractive summarization** — Summarization where the model can produce text not in the input.
- **Encoder cross-attention** — The decoder's mechanism for attending to encoder hidden states during generation.
- `model.generate()` — The autoregressive token-generation entry point.
- **Beam search** — Deterministic generation algorithm that keeps the top-N candidate sequences.
- `do_sample=False` / `num_beams=4` — Standard summarization-evaluation generation settings.
- **Exact-match (EM)** — QA metric: 1 if normalized prediction equals normalized gold, else 0.
- **Token-F1** — QA metric: F1 over bag-of-tokens after normalization.
- **Answer normalization** — Lowercase + strip articles + strip punctuation + collapse whitespace; required pre-processing for QA metrics.
- **ROUGE-1 / ROUGE-2 / ROUGE-L** — Summarization F1 metrics over unigrams / bigrams / longest common subsequence.
- **Faithfulness** — Whether a generated summary is factually consistent with the source. **Not captured by ROUGE.**
- **Pre-trained inference** — Using a pre-trained model directly without fine-tuning.
- **Fine-tuning vs. pre-trained inference trade-off** — The cost/quality decision the M7 deliverable asks you to articulate.

Summary

- A `pipeline()` is the high-level inference abstraction in Hugging Face — tokenizer + model + post-processing in one callable. Use it for inference on standard tasks; drop down to `AutoModel` when you need finer control.
- Extractive QA is **span prediction** over an encoder's hidden states. The answer is always a contiguous substring of the context. Encoder-family models host the QA head.
- Evaluate extractive QA with **exact-match (EM)** and **token-F1**, both after the standard normalization (lowercase, strip articles, strip punctuation, collapse whitespace).
- Abstractive summarization is **conditional generation** over an encoder–decoder. Use deterministic beam search (`do_sample=False`, `num_beams=4`) for evaluation reproducibility.
- Evaluate summarization with **ROUGE-1, ROUGE-2, ROUGE-L F1**. ROUGE catches lexical overlap; it does **not** catch faithfulness, coherence, or reading order. Supplement with qualitative analysis for high-stakes use.
- The fine-tuning vs. pre-trained inference decision rests on five factors: labeled-data availability, task specificity, compute budget, iteration speed, quality target. The Integration Task asks you to make this decision concrete with measured numbers.

What's Next

- **Tonight (Day 1, Sunday evening):** Core Skills Drill — build pipelines and metrics on a fixture.
- **Tomorrow (Day 2, Monday):** Applied Lab — pre-trained QA on the curated tech-news QA set, full evaluation harness.
- **Wednesday (Day 4):** Integration Task — pre-trained summarization on the tech news corpus + the integrated evaluation report (the M7 deliverable).
- **Thursday (Day 5):** Peer review of one teammate's Lab 7B QA evaluation report; reflection; readiness check; integration submission.

Module 8 begins T11 (Sun 31 May 2026). M8 wraps the QA pipeline behind a retriever (Weaviate) — RAG.

© 2026 LevelUp Economy. All rights reserved. Unauthorized reproduction or distribution of this material is prohibited.